(19)中华人民共和国国家知识产权局

(12)发明专利

(10)授权公告号 CN 104166666 B
(45)授权公告日 2017.03.08

(21)申请号 201410207285.6

(22)申请日 2014.05.15

(65)同一申请的已公布的文献号
申请公布号 CN 104166666 A

(43)申请公布日 2014.11.26

(73)专利权人 杭州斯凯网络科技有限公司
地址 310013 浙江省杭州市西湖区紫荆花路2号联合大厦10楼

(72)发明人 周正中

(74)专利代理机构 杭州杭诚专利事务所有限公司 33109
代理人 尉伟敏

(51)Int.Cl.
G06F 17/30(2006.01)
G06Q 10/06(2012.01)

(56)对比文件
CN 101960454 A,2011.01.26,
CN 102467559 A,2012.05.23,
CN 103412933 A,2013.11.27,
龙晓泉等.《实时数据仓库的一种实现方法》.《计算机系统应用》.2010,第19卷(第6期),正文第178-182页.

审查员 高丹丹

权利要求书1页 说明书15页

(54)发明名称

PostgreSQL高并发流式大数据多维度准实时统计的方法

(57)摘要

本发明涉及PostgreSQL高并发流式大数据多维度准实时统计的方法。其技术方案要点是：创建流水表增量状态表、统计函数、函数增量状态表和分析函数，对流水表增量状态表函数增量状态表进行初始化；计算分析函数取数的次数，以整数方式记录，作为统计次数数据，分析函数工作，进入子步骤，计算机根据统计函数得出的流水统计数据输出含有多维度准实时统计数据的统计维度表。本发明节约硬件投入至少100倍，实时时效控制在1分钟以内。

CN 104166666 B

1.一种PostgreSQL 高并发流式大数据多维度准实时统计的方法，适用于OLTP系统，其特征在于：

步骤一：创建流水表增量状态表、统计函数、函数增量状态表和分析函数，对流水表增量状态表、函数增量状态表进行初始化；

步骤二：计算分析函数取数的次数，以整数方式记录，作为统计次数数据，

步骤三：分析函数工作，进入子步骤，

子步骤一：计算机判断取数据的参数值是否正确，如果正确将数据录入流水信息表，如果不正确则结束本方法并报错，

子步骤二：从函数增量状态表中取出统计函数，存储在计算机内存中，遍历统计函数的函数名和各变量，

子步骤三：把未执行事务号存入内存中，

子步骤四：在流水表增量状态表中取出历史截止时间值，

子步骤五：将历史未执行事务号与未执行事务号做集合加运算，

子步骤六：从流水表中根据统计次数数据取出大于等于历史截止时间值的事务号结果值，所述事务号结果值包括历史截止时间值和第一部分流水数据，历史截止时间值根据事务号结果值进行更新，

子步骤七：计算机从流水表中先对未执行事务号和历史未执行事务号进行集合差运算获得差异值，再对未执行事务号和历史未执行事务号进行取交集运算获得交集值，将交集值与差异值做集合与运算得出第二部分流水数据，

子步骤八：将第一部分流水数据和第二部分流水数据合并得出流水数据，将流水数据更新至流水表增量状态表，将历史截止时间值也更新至流水表增量状态表中，遍历统计函数中的变量后，更新函数增量状态表，

子步骤九：流水数据由统计函数调用，执行步骤四，

步骤四：计算机根据统计函数得出的流水统计数据输出含有多维度准实时统计数据的统计维度表；

所述流水表为OLTP系统固有组成。

2.根据权利要求1所述的PostgreSQL 高并发流式大数据多维度准实时统计的方法，其特征在于：当取增量数据消耗硬件成本更高时，使用所有维度统一增量数据集方式统计，在数据统计消耗硬件成本更高时，使用维度独立增量数据集方式统计。

3.根据权利要求2所述的PostgreSQL 高并发流式大数据多维度准实时统计的方法，其特征在于：使用所有维度统一增量数据集方式统计，所有统计维度表对应同一份流水表增量状态表和同一份函数增量状态表。

PostgreSQL高并发流式大数据多维度准实时统计的方法

技术领域

[0001] 本发明属于一种数据统计方法，特别涉及PostgreSQL高并发流式大数据多维度准实时统计的方法。

背景技术

[0002] 随着互联网的发展，互联网上的应用和用户越来越多，用户产生的数据也爆发性的增长，通过用户产生的数据，可以按照数据的各个维度分析得出企业关注的数据，例如用户的流动性，差异性，传播性，产品和用户群体的关系，应用热度分时，分地域的排行等等，根据用户特性挖掘用户潜在的需求等；程序获取用户产生的数据最简单有效的方法就是流水式数据，单个数据包里面包含了发生时间点的各个维度的所有信息量，这种场景的特性是并发量大，数据量大，因此对时效要求比较高的数据分析来说是一个非常巨大的挑战。

[0003] 传统的做法有几种：

[0004] 由于OLTP系统的计算资源和IO资源的局限性，直接在OLTP系统上做多维度的统计是不合理的，第一种解决办法是将这种流式数据从OLTP系统异步的写入数据仓库，而数据仓库并不适合做实时的统计，比较适合做离线(如隔夜)的分析，无法满足时效高的场景需求。同时数据仓库的基础设施以及软件初期投入成本高昂，耗资往往是OLTP系统的百倍以上。

[0005] 另一种方法是在OLTP系统上使用触发器，在触发器函数中包含需要统计的维度的业务逻辑，实时的更新统计结果，这种方法适合小数据量的场景，因为每条数据都触发，显然带来了比较庞大的计算开销，极其容易产生写入瓶颈，无法满足高并发和大数据场景下的高时效统计需求。

[0006] 申请号：201010551123。6，公开了一种多层次多维度的数据属性分析方法及装置。其特征是：建立维护公用基础资料数据的公用代码平台，所述公用基础资料包括基础资料和/或公用代码表；建立所述基础资料数据分析的数据关系模型作为基础资料分析平台；根据所述基础资料分析平台的数据关系进行数据属性多层次与多维度分析。其技术方案与现有技术一样，存在同时数据仓库的基础设施以及软件初期投入成本高昂，耗资往往是OLTP系统的百倍以上的问题。

发明内容

[0007] 本发明解决了上述技术存在同时数据仓库的基础设施以及软件初期投入成本高昂，耗资往往是OLTP系统的百倍以上。或是因为每条数据都触发，显然带来了比较庞大的计算开销，极其容易产生写入瓶颈，无法满足高并发和大数据场景下的高时效统计需求的问题，提供一种克服现有技术缺点，的PostgreSQL高并发流式大数据多维度准实时统计的方法。

[0008] 本发明解决其技术问题所采用的技术方案是：一种PostgreSQL高并发流式大数据多维度准实时统计的方法，适用于OLTP系统，

[0009] 步骤一：创建流水表增量状态表、统计函数、函数增量状态表和分析函数，对流水表增量状态表函数增量状态表进行初始化；

[0010] 步骤二：计算分析函数取数的次数，以整数方式记录，作为统计次数数据，

[0011] 步骤三：分析函数工作，进入子步骤，

[0012] 子步骤一：计算机判断取数据的参数值是否正确，如果正确将数据录入流水信息表，如果不正确则结束本方法并报错，

[0013] 子步骤二：从函数增量状态表中取出统计函数，存储在计算机内存中，遍历统计函数的函数名和各变量，

[0014] 子步骤三：把未执行事务号存入内存中，

[0015] 子步骤四：在流水表增量状态表中取出历史截止时间值，

[0016] 子步骤五：将历史未执行事务号与未执行事务号做集合加运算，

[0017] 子步骤六：从流水表中根据统计次数数据取出大于等于历史截止时间值的事务号结果值，所述事务号结果值包括历史截止时间值和第一部分流水数据，历史截止时间值根据事务号结果值进行更新，

[0018] 子步骤七：计算机从流水表中先对未执行事务号和历史未执行事务号进行集合差运算获得差异值，再对未执行事务号和历史未执行事务号进行取交集运算获得交集值，将交集值与差异值做集合与运算得出第二部分流水数据，

[0019] 子步骤八：将第一部分流水数据和第二部分流水数据合并得出流水数据，将流水数据更新至流水表增量状态表，将历史截止时间值也更新至流水表增量状态表中，遍历统计函数中的变量后，更新函数增量状态表，

[0020] 子步骤九：流水数据由统计函数调用，执行步骤四，

[0021] 步骤四：计算机根据统计函数得出的流水统计数据输出含有多维度准实时统计数据的统计维度表；

[0022] 所述流水表为OLTP系统固有组成。

[0023] 相对传统方法通过大量基础设施以及软件投资提高计算资源来解决大数据的准实时统计需求，性价比很低，往往不是创业型或者中小企业所能承担的。另一些现有技术则通过OLTP中创建触发器来实现实时的统计，受制于请求量，无法满足高并发的请求。采用本发明的方法很好的解决了三个问题，一是成本，二是高并发，三是时效；使得创业型或中小企业在可以接受的成本投入下，也能实现大数据的准实时数据挖掘，相对成本问题，由于本发明不需要OLTP以外额外的设备投入，所以不会带来成本的提升；

[0024] 针对高并发的问题，本发明采用异步的方式解决了高并发的问题，增量对数据进行统计，同时不影响一致性和高并发；针对时效问题，本发明支持各维度并行调度，可以提高统计的并发度，很好的解决了串行统计带来的时效问题。经测试，采用本发明的方法进行高并发流式大数据多维度准实时统计与传统方法相比，在不影响并发的前提下，节约硬件投入至少100倍，实时时效控制在1分钟以内，实际测试约5秒。相比数据仓库的隔天报表时效提高1440倍以上，统计性能比直接统计性能提升1万倍以上。

[0025] 作为优选，当取增量数据消耗硬件成本更高时，使用所有维度统一增量数据集方式统计，在数据统计耗硬件成本更高时，使用维度独立增量数据集方式统计。

[0026] 这个设定可以根据人工设定或者采用计算机自动获取之后的计算获得，这样设置，针对不同的硬件条件有不同的对应方法，确定了相应的最优方式。

[0027] 作为优选，使用所有维度统一增量数据集方式统计，所有统计维度表对应同一份

流水表增量状态表和同一份函数增量状态表。

[0028] 作为优选，使用所有维度统一增量数据集方式统计，每份统计维度表均对应有一份流水表增量状态表和同一份函数增量状态表。

[0029] 作为优选，在所述子步骤二中，在获取统计维度信息前，系统对均获取一个用于确保统计逻辑数据的一致性的adv锁，在所述子步骤九中增加释放adv锁的步骤。

[0030] 本发明的实质性效果是：采用本发明的方法进行高并发流式大数据多维度准实时统计与传统方法相比，在不影响并发的前提下，节约硬件投入至少100倍，实时时效控制在1分钟以内，实际测试约5秒。相比数据仓库的隔天报表时效提高1440倍以上，统计性能比直接统计性能提升1万倍以上。

具体实施方式

[0031] 下面通过具体实施例，对本发明的技术方案作进一步的具体说明。

[0032] 实施例：

[0033] 一种PostgreSQL高并发流式大数据多维度准实时统计的方法，适用于OLTP系统，

[0034] 步骤一：创建流水表增量状态表、统计函数、函数增量状态表和分析函数，对流水表增量状态表函数增量状态表进行初始化；

[0035] 步骤二：计算分析函数取数的次数，以整数方式记录，作为统计次数数据，

[0036] 步骤三：分析函数工作，进入子步骤，

[0037] 子步骤一：计算机判断取数据的参数值是否正确，如果正确将数据录入流水信息表，如果不正确则结束本方法并报错，

[0038] 子步骤二：从函数增量状态表中取出统计函数，存储在计算机内存中，遍历统计函数的函数名和各变量，

[0039] 子步骤三：把未执行事务号存入内存中，

[0040] 子步骤四：在流水表增量状态表中取出历史截止时间值，

[0041] 子步骤五：将历史未执行事务号与未执行事务号做集合加运算，

[0042] 子步骤六：从流水表中根据统计次数数据取出大于等于历史截止时间值的事务号结果值，所述事务号结果值包括历史截止时间值和第一部分流水数据，历史截止时间值根据事务号结果值进行更新，

[0043] 子步骤七：计算机从流水表中先对未执行事务号和历史未执行事务号进行集合差运算获得差异值，再对未执行事务号和历史未执行事务号进行取交集运算获得交集值，将交集值与差异值做集合与运算得出第二部分流水数据，

[0044] 子步骤八：将第一部分流水数据和第二部分流水数据合并得出流水数据，将流水数据更新至流水表增量状态表，将历史截止时间值也更新至流水表增量状态表中，遍历统计函数中的变量后，更新函数增量状态表，

[0045] 子步骤九：流水数据由统计函数调用，执行步骤四，

[0046] 步骤四：计算机根据统计函数得出的流水统计数据输出含有多维度准实时统计数据的统计维度表；

[0047] 所述流水表为OLTP系统固有组成。

[0048] 当取增量数据消耗硬件成本更高时，使用所有维度统一增量数据集方式统计，在

数据统计耗硬件成本更高时，使用维度独立增量数据集方式统计。

[0049] 使用所有维度统一增量数据集方式统计，所有统计维度表对应同一份流水表增量状态表和同一份函数增量状态表。

[0050] 使用所有维度统一增量数据集方式统计，每份统计维度表均对应有一份流水表增量状态表和同一份函数增量状态表。

[0051] 在所述子步骤二中，在获取统计维度信息前，系统对均获取一个用于确保统计逻辑数据的一致性的adv锁，在所述子步骤九中增加释放adv锁的步骤。

[0052] 以实际例子来讲解具体的实施过程为：

[0053] 首先要创建流水表，在流水表中，以xid作为增量截止标识，另外需要xid_snapshot作为气泡标记.

[0054]
```
create table log
(
  id serial primary key,
  xid int8 default txid_current() not null,
  c1 int not null,
  c2 int not null,
  c3 int not null,
  c4 text not null,
  crt_time timestamp default now()
);
```

[0055] 为了加快取数据的速度，在xid上加上必要的索引.

[0056] create index idx_log_1on log(xid)；

[0057] 创建维度统计表，本例以c1一个维度，c2，c3一个维度，加上日，周，月，年总共8个维度.

[0058]
```
create table log_c1_cnt_day (c1 int, cnt int8, stat_time text, primary key(c1,stat_time));
create table log_c1_cnt_week (c1 int, cnt int8, stat_time text, primary key(c1,stat_time));
create table log_c1_cnt_month (c1 int, cnt int8, stat_time text, primary key(c1,stat_time));
create table log_c1_cnt_year (c1 int, cnt int8, stat_time text, primary key(c1,stat_time));
```

```
create table log_c2_c3_cnt_day (c2 int, c3 int, cnt int8, stat_time text, primary key(c2,c3,stat_time));
create table log_c2_c3_cnt_week (c2 int, c3 int, cnt int8, stat_time text, primary key(c2,c3,stat_time));
create table log_c2_c3_cnt_month (c2 int, c3 int, cnt int8, stat_time text, primary key(c2,c3,stat_time));
create table log_c2_c3_cnt_year (c2 int, c3 int, cnt int8, stat_time text, primary key(c2,c3,stat_time));
```

[0059] 创建分析注册表，记录每个明细表每次分析的截止xid，xip.

[0060]
```
create table log_read
(
tablename name not null,
xid int8 not null, -- xid 记录增量统计的截止 xid
xip int8[], -- xip 记录当前活动事务，不计入当前统计范畴，避免气泡问题
xip_res int8[], -- 用于与 xid 比对的数据，保留所有>=xid 的 xip 信息.
```

[0061]
```
mod_time timestamp,
primary key (tablename)
);
```

[0062] 插入初始记录,表的初始记录xid取值范围(>=0and<=txid_snapshot_xmin-1)

[0063] insert into log_read values('log',0,null,null,now());

[0064] 创建分析维度信息注册表,记录每个维度的初始xid信息.

[0065] 因为所有维度使用同一个表级别的截至值(log_read.xid),但是维度可能是后期加入的,所以初始xid可能不一样,每个维度一条记录信息.

[0066]
```
create table log_read_func (
  tablename name not null,
  func text not null, -- 该维度调用的函数
  init_xid int8, -- 初始值由 analyze_log 计算并更新，历史数据的统计交给其他进程来处理，历史数据将统计到 <=init_xid and 不包含 init_xip.
  init_xip int8[], -- 初始值由 analyze_log 计算并更新.
  isinit boolean not null, -- true 标识这个维度是初始的，false 标识不是初始的.
  info text, -- 备注
  primary key (tablename, func)
);
alter table log_read_func add constraint fk_log_read_func_1 foreign key(tablename) references log_read(tablename);

插入维度初始记录，stat_log_c1 函数是用于统计 C1 维度的函数.
insert into log_read_func(tablename,func,init_xid,init_xip,isinit,info)
  values('log','stat_log_c1',null,null,true,null);
```

[0067] 创建log表的数据分析函数,统一取增量数据,然后调用维度分析函数.

[0068] v_limit用于限制多少个xid,而不是多少条记录,当一个XID有多条记录时,如果取到这个XID的话,这个XID的数据也会全部被处理.

[0069] 因为使用v_limit来限定行数,就会出现比较危险的情况,因为如果一个事务包含多条记录的话,限定行数的方法可能导致一个事务的数据只取到中间部分,而剩余部分下次就取不到了.

[0070]

```
create or replace function analyze_log(v_limit int) returns void as $$
declare
  v_advisory_xact_lock int8 := null;  -- 串行处理锁.

  v_xid_snap txid_snapshot := null;  -- 当前事务状态快照
  v_xmin int8 := null;  -- 当前事务状态快照中未完成的最小事务
  v_xmax int8 := null;  -- 当前事务状态快照中未分配的最小事务

  v_xip int8[] := null;  -- 当前事务状态快照中未完成的事务数组

  v_func_agg text[];  -- 统计维度函数数组
  v_func text;  -- 统计维度函数

  v_log_read_log_xid int8 := null;  -- 上次log的xid分析截止位
  v_log_read_log_xid_update int8 := null;  -- 更新值, 不能为空

  v_log_read_log_xip int8[] := null;  -- 上次log_read.xip(tablename=log)
  v_log_read_log_xip_do  int8[]  :=  null;  -- 解析本次
log_read.xip(tablename=log) where (xip !@ txid_snapshot)
  v_log_read_log_xip_update int8[] := null;  -- xip更新值
  v_log_read_log_xip_res int8[] := null;  -- xip保留值
  v_log_read_log_xip_res_update int8[] := null;  -- xip保留更新值, 所有大于
v_log_read_log_xid_update的元素必须保留.

  v_log log[] := null;  -- 聚合本次log的分析数组, [末尾调用,false]
  v_log_doxip log[] := null;  -- 聚合本次分析log数组:
                          -- where log.xid (@ log_read.xip(tablename=log)
and !@ txid_snapshot) , [末尾调用,false]
```

[0071]

```
  v_log_read_func_xid int8;  -- 本次取出的 v_log 中的最小 xid-1, 没有则取
OLD.log_read.xid. [用于更新 log_read_func.init_xid where isinit=true]
  v_log_read_func_xip int8[];  -- (OLD.log_read.xip + v_xip) ; [用于更新
log_read_func.init_xip where isinit=true]

begin
  -- 判断 limit
  if v_limit <=0 then
    raise notice 'please ensure v_limit > 0 .';
    return;
  end if;

  -- 串行处理, 如果不能获得锁则直接退出. 确保 v_advisory_xact_lock 全局唯一.
  v_advisory_xact_lock := 1;
  if not pg_try_advisory_xact_lock(v_advisory_xact_lock) then
    raise notice 'Another function is calling, this call will exit.';
    return;
  end if;

  -- 生成统计维度, 没有则直接退出
  perform 1 from log_read_func where tablename='log' limit 1;
```

```
  if not found then
    raise notice 'No func in log_read_func with tablename:%.', 'log';
    return;
  else
    select array_agg(func) into v_func_agg from log_read_func where
tablename='log';
  end if;

  -- 生成 xid snapshot 数据.
  v_xid_snap := txid_current_snapshot();
  v_xmin := txid_snapshot_xmin(v_xid_snap);
  v_xmax := txid_snapshot_xmax(v_xid_snap);
  select array_agg(t) into v_xip from txid_snapshot_xip(v_xid_snap) g(t);

  -- 取 v_log_read_log_xid 截止值, v_log_read_log_xip 数组.
  select xid,xip,xip_res into
v_log_read_log_xid,v_log_read_log_xip,v_log_read_log_xip_res from log_read
where tablename='log';
  if not found then
    raise notice 'log_read no log entry, please add it in log_read table first.';
    return;
  end if;

  -- 生成 v_log_read_func_xip.
  -- 必须放在更新 log_read 之前, 否则取到的就是更新后的数据了.
  v_log_read_func_xip := array_cat(v_log_read_log_xip, v_xip);

  -- 取 log1(取非 xip 中的数据, 隔离 log2 操作)
  -- 取 xid 临界点
  select max(xid) into v_log_read_log_xid_update from (select xid from log
where xid > v_log_read_log_xid and xid < v_xmax and xid not in (select * from
unnest(v_xip) union all select * from unnest(v_log_read_log_xip_res)) order by
xid limit v_limit) t;
  if v_log_read_log_xid_update is not null then
    raise notice '取 log1';
    -- 根据临界点, 取 log 数据
    select array_agg(log) into v_log from (select log from log where xid >
v_log_read_log_xid and xid<=v_log_read_log_xid_update and xid not in (select
* from unnest(v_xip) union all select * from unnest(v_log_read_log_xip_res))
order by xid) t;
    -- 如果有数据, v_log_read_func_xid 取 v_log.min(xid) - 1
    select min(i.xid) - 1 into v_log_read_func_xid from unnest(v_log) i;
```

[0072]

[0073]

```
  else
    -- 如果没有数据，更新仍不变
    v_log_read_log_xid_update := v_log_read_log_xid;
    -- 如果没有数据，v_log_read_func_xid取OLD.log_read.xid
    v_log_read_func_xid := v_log_read_log_xid;
  end if;

  -- 取log2 (log_xip - v_xip) (取xip中的数据，隐离log1操作)
    -- 生成log_read.xip(tablename=log) do数组(已经完成的事务)
  select array_agg(i) into v_log_read_log_xip_do from (select * from
unnest(v_log_read_log_xip) i except select * from unnest(v_xip))t where i is
not null;
  -- 生成log_read.xip(tablename=log) update数组(未完成的事务)
  select array_agg(i) into v_log_read_log_xip_update from
  ( select i from (select * from unnest(v_log_read_log_xip) i union all select
* from unnest(v_xip)
    except select * from unnest(v_log_read_log_xip_do)) t where i is not null
group by i ) t;
  -- 生成xip_res更新值
  select array_agg(i) into v_log_read_log_xip_res_update from (select * from
unnest(v_log_read_log_xip_res) i union select * from unnest(v_log_read_log_xip)
union select * from unnest(v_xip))t where i>v_log_read_log_xid_update;
  -- 生成log do数组
  select array_agg(log) into v_log_doxip from log where xid in (select * from
unnest(v_log_read_log_xip_do));

  -- 更新log_read(tablename=log)
  update log_read set
    xip=v_log_read_log_xip_update,
    xid=v_log_read_log_xid_update,
    xip_res=v_log_read_log_xip_res_update,
    mod_time=now()
  where tablename='log';
  -- raise notice 'log_read.oldxip(log): %', v_log_read_log_xip;
  -- raise notice 'log_read.newxip(log): %', v_log_read_log_xip_update;
  --       raise       notice       'log_read.newxiprcs(log):       %',
v_log_read_log_xip_res_update;

  -- 分析函数可以另外写，在此调用。
  foreach v_func in array v_func_agg loop
    -- 更新log_read_func where isinit=true
    update log_read_func set
```

```
      init_xid=v_log_read_func.xid,
      init_xip=v_log_read_func.xip,
      isinit=false
      where tablename='log'
      and func=v_func
      and isinit;
    -- 执行统计函数
    execute 'select '||v_func||'($1)' using v_log;
    execute 'select '||v_func||'($1)' using v_log.doxip;
  end loop;

return;
end;
$$ language plpgsql;
```

创建统计函数 stat_log_cl

[0074]

```
CREATE OR REPLACE FUNCTION public.stat_log_cl(v_log log[])
 RETURNS void
 LANGUAGE plpgsql
AS $function$
declare
  v_stat_time text;
  v_cl int;
  v_cnt int8;
begin
  -- 统计 log cl cnt day
  for v_stat_time, v_cl, v_cnt in select to_char(crt_time, 'yyyymmdd'), cl ,
count(*) from (select ((unnest(v_log)::log)).*) t group by to_char(crt_time,
'yyyymmdd'), cl loop
    perform 1 from log_cl_cnt_day where cl=v_cl and stat_time=v_stat_time;
    if not found then
      insert into log_cl_cnt_day(cl, cnt, stat_time) values (v_cl, v_cnt,
v_stat_time);
    else
      update log_cl_cnt_day set cnt=cnt+v_cnt where cl=v_cl and
stat_time=v_stat_time;
    end if;
  end loop;
end;
$function$;
```

[0075] 设置统计维度的初始记录开始位置，xid＝0表示表里的所有数据都需要统计.

[0076]

```
update log_read set xid=0, xip=null, xip_res=null;
update log_read_func set isinit=true;
```

[0077] 使用pgbench模拟并发数据录入，录入语句中包括回滚的场景，更贴近实际应用场景：

```
vi ins.sql
begin;
insert into log (c1,c2,c3,c4) values(round(random()*10),1,2,3);
insert into log (c1,c2,c3,c4) values(round(random()*10),1,2,3);
end;
```

[0078]

```
begin;
insert into log (c1,c2,c3,c4) values(round(random()*10),1,2,3);
insert into log (c1,c2,c3,c4) values(round(random()*10),1,2,3);
rollback;
insert into log (c1,c2,c3,c4) values(round(random()*10),1,2,3);
insert into log (c1,c2,c3,c4) values(round(random()*10),1,2,3);
```

[0079] 执行以下进程,数据录入10分钟.

[0080]

```
pgbench -M prepared -f ./ins.sql -r -n -h $PGDATA -U postgres -T 600 -c 4 -j
2
```

[0081] 摘录测试结果如下,5000tps,30000qps.

[0082]

```
pg931@db-172-16-3-150-> pgbench -M prepared -f ./ins.sql -r -n -h $PGDATA -U
postgres -T 600 -c 4 -j 2
transaction type: Custom query
scaling factor: 1
query mode: prepared
number of clients: 4
number of threads: 2
duration: 600 s
number of transactions actually processed: 3015974
tps = 5026.614428 (including connections establishing)
tps = 5026.668351 (excluding connections establishing)
statement latencies in milliseconds:
        0.035321        begin;
        0.100925                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.098142                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.047806        end;
        0.032409        begin;
        0.100818                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
```

[0083]

```
        0.098428                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.047868        rollback;
        0.112753                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.113537                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
```

[0084] 同时开启以下统计程序,每隔6秒统计一次,每次最多取100万个事务的增量数据.

```
vi analyze.sh
#!/bin/bash
for ((i=0;i<101;i++))
do
```

[0085]

```
psql -U postgres -c "select * from analyze_log(1000000);"
sleep 6
done
chmod 500 analyze.sh
./analyze.sh
```

[0086] 统计比数据录入结束延迟约5秒.

[0087] 测试结束后验证统计数据是否准确,比对流水表和统计维度表的数据是否一致.

[0088]

```
digoal=# \timing
Timing is on.
digoal=# select c1,count(*),to_char(crt_time,'yyyymmdd') from log group by
c1,to_char(crt_time,'yyyymmdd') order by c1;
 c1 |  count  | to_char
----+---------+----------
  0 |  603051 | 20140224
  1 | 1205493 | 20140224
  2 | 1208338 | 20140224
  3 | 1206984 | 20140224
  4 | 1204773 | 20140224
  5 | 1206976 | 20140224
  6 | 1206799 | 20140224
  7 | 1207715 | 20140224
  8 | 1206041 | 20140224
  9 | 1205446 | 20140224
 10 |  602280 | 20140224
(11 rows)
```

[0089]

```
Time: 78078.999 ms
digoal=# select * from log_c1_cnt_day where cnt<>0 order by c1;
 c1 |   cnt   | stat_time
----+---------+----------
  0 |  603051 | 20140224
  1 | 1205493 | 20140224
  2 | 1208338 | 20140224
  3 | 1206984 | 20140224
  4 | 1204773 | 20140224
  5 | 1206976 | 20140224
  6 | 1206799 | 20140224
  7 | 1207715 | 20140224
  8 | 1206041 | 20140224
  9 | 1205446 | 20140224
 10 |  602280 | 20140224
(11 rows)

Time: 0.973 ms
```

[0090] 统计表和流水表的比对结果一致，性能提升80000多倍.

[0091] 新增统计维度的测试，加入需要新增统计维度，可以通过以下方法添加统计维度

[0092] 创建新维度的统计计算函数stat_log_c1_week.

[0093]

```
CREATE OR REPLACE FUNCTION public.stat_log_c1_week(v_log log[])
 RETURNS void
 LANGUAGE plpgsql
AS $function$
declare
  v_stat_time text;
  v_c1 int;
  v_cnt int8;
begin
  -- 统计 log_c1_cnt_week
  for v_stat_time, v_c1, v_cnt in select to_char(date(crt_time)-(EXTRACT(ISODOW FROM date(crt_time)))::int+1,'yyyymmdd'), c1, count(*) from (select ((unnest(v_log)::log)).*) t group by to_char(date(crt_time)-(EXTRACT(ISODOW FROM date(crt_time)))::int+1,'yyyymmdd'), c1 loop
    perform 1 from log_c1_cnt_week where c1=v_c1 and stat_time=v_stat_time;
    if not found then
      insert into log_c1_cnt_week(c1, cnt, stat_time) values (v_c1, v_cnt, v_stat_time);
    else
```

[0094]
```
        update log_c1_cnt_week set cnt=cnt+v_cnt where c1=v_c1 and
stat_time=v_stat_time;
      end if;
    end loop;
end;
$function$;
```

[0095] 注册统计维度函数,注册时isinit=true.第一次调用analyze_log后更新为false,同时更新init_xid,init_xip等.

[0096]
```
insert into log_read_func(tablename,func,init_xid,init_xip,isinit,info)
values('log','stat_log_c1_week',null,null,true,null);
```

[0097] 继续使用pgbench数据录入测试60秒

[0098]
```
pg931@db-172-16-3-150-> pgbench -M prepared -f ./ins.sql -r -n -h $PGDATA -U
postgres -T 60 -c 8 -j 2
transaction type: Custom query
scaling factor: 1
query mode: prepared
number of clients: 8
number of threads: 2
duration: 60 s
number of transactions actually processed: 389007
tps = 6483.366365 (including connections establishing)
tps = 6484.709493 (excluding connections establishing)
statement latencies in milliseconds:
        0.066300        begin;
        0.147173                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.148250                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.088911        end;
        0.062321        begin;
        0.148291                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.147312                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.088264        rollback;
        0.162848                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
        0.164027                insert into log (c1,c2,c3,c4)
values(round(random()*10),1,2,3);
```

[0099] 同时开启analyze.sh进行增量统计

[0100] ./analyze.sh

[0101] 分析函数运行完后,比对数据一致性,结果一致.

```
digoal=# select * from log_read;
 tablename |    xid    | xip | xip_res |          mod_time
-----------+-----------+-----+---------+----------------------------
 log       | 13622239  |     |         | 2014-02-24 15:10:50.653221
(1 row)
digoal=# select * from log_read_func where func='stat_log_c1_week';
 tablename |         func          | init_xid                    init_xip
 | isinit | info
-----------+-----------------------+----------+-----------------------------
-----+--------+------
 log                                | stat_log_c1_week      |     12066206      |
{12585180,12585191,12585192,12585194} | f      |
(1 row)
```

比对第一个维度的数据 :

```
digoal=# select c1,count(*),to_char(crt_time,'yyyymmdd') from log where xid>0
group by c1,to_char(crt_time,'yyyymmdd') order by c1;
 c1 |  count  | to_char
----+---------+----------
  0 |  680448 | 20140224
  1 | 1360117 | 20140224
  2 | 1363871 | 20140224
  3 | 1363599 | 20140224
  4 | 1360661 | 20140224
  5 | 1362271 | 20140224
  6 | 1363122 | 20140224
  7 | 1363912 | 20140224
  8 | 1361196 | 20140224
  9 | 1361391 | 20140224
 10 |  679336 | 20140224
(11 rows)
Time: 90814.411 ms
digoal=# select * from log_c1_cnt_day where cnt<>0 order by c1;
 c1 |  cnt    | stat_time
----+---------+-----------
  0 |  680448 | 20140224
  1 | 1360117 | 20140224
  2 | 1363871 | 20140224
  3 | 1363599 | 20140224
```

[0102]

```
  4 | 1360661 | 20140224
  5 | 1362271 | 20140224
  6 | 1363122 | 20140224
  7 | 1363912 | 20140224
  8 | 1361196 | 20140224
  9 | 1361391 | 20140224
 10 |  679336 | 20140224
(11 rows)
Time: 0.896 ms
```

[0103]

[0104] 比对第二个维度的数据，由于第二个维度是后期加入的，所以统计时需要排除掉历史数据.

[0105]
```
digoal=# select c1,count(*),to_char(date(crt_time)-(EXTRACT(ISODOW FROM date(crt_time)))::int+1,'yyyymmdd') from log where xid>12066206 or xid in (12585180,12585191,12585192,12585194) group by c1,to_char(date(crt_time)-(EXTRACT(ISODOW FROM date(crt_time)))::int+1,'yyyymmdd') order by c1;
 c1 | count | to_char
----+-------+----------
  0 | 77397 | 20140224
  1 | 154624 | 20140224
  2 | 155533 | 20140224
  3 | 156615 | 20140224
  4 | 155888 | 20140224
  5 | 155295 | 20140224
  6 | 156323 | 20140224
  7 | 156197 | 20140224
  8 | 155155 | 20140224
  9 | 155945 | 20140224
 10 | 77056 | 20140224
(11 rows)
Time: 11739.503 ms
digoal=# select * from log_c1_cnt_week where cnt<>0 order by c1;
 c1 | cnt | stat_time
----+-------+----------
  0 | 77397 | 20140224
  1 | 154624 | 20140224
  2 | 155533 | 20140224
  3 | 156615 | 20140224
  4 | 155888 | 20140224
  5 | 155295 | 20140224
  6 | 156323 | 20140224
  7 | 156197 | 20140224
```

[0106]
```
  8 | 155155 | 20140224
  9 | 155945 | 20140224
 10 | 77056 | 20140224
(11 rows)
Time: 0.719 ms
```

[0107] 以上所述的实施例只是本发明的一种较佳的方案，并非对本发明作任何形式上的限制，在不超出权利要求所记载的技术方案的前提下还有其它的变体及改型。